Informe Proyecto Agente Inversor de Deep Reinforcement Learning (DQN) para Trading Financiero

Materia: **Simulación y Aprendizaje por Refuerzo MIA 2026-5**
Profesor: **Emilio Muñoz Perez**

Presentado por (GRUPO 1) :
- Gloria Yaned Leguizamon Arboleda
- Ricardo Sandoval Chaves
- Edwin Aviles Cogollo
- Orly Alberto Alvao Barajas
- Edgar Julian Mendez Ortegon
- Saud Mauricio Gonzalez Rodriguez
- Luis Alfonso Fajardo Andrade

Repositorio GitHub Proyecto: https://github.com/ricardoschmg/trading-rl-project

De acuerdo al Agende implementado se tienen las siguientes consideraciones:

1. Introducción y Formulación del Problema (Estado, Acciones y Entorno)

En este proyecto nos propusimos un reto clásico de Aprendizaje por Refuerzo aplicado a finanzas: Construir un agente inteligente capaz de tomar decisiones de inversión (comprar, vender o mantener) sobre un activo histórico (en nuestro caso, acciones de Tesla - TSLA) utilizando **Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning)**.

Para lograrlo, se definió el entorno de la siguiente manera:

- **El Estado (Observación):** Para evitar que la red neuronal se confunda con números Para evitar que escalas numéricas dispares confundan al agente, la información se normaliza.
- **Estado (Observación):** Un vector de 3 dimensiones ***[price_change, normalized_balance, current_position]*** que representa:
    - **Retorno porcentual diario del precio:** Variación del precio de cierre respecto al día anterior.
    - **Balance normalizado:** Efectivo disponible dividido entre el capital inicial ($10,000).
    - **Posición actual:** Proporción del portafolio actualmente invertida.
- **Espacio de Acciones:** Un espacio discreto *Discrete(3)* que indica el porcentaje de capital objetivo a invertir:
    - **0**: 0% invertido (Vender todo / Mantener 100% efectivo).

- **1**: 50% invertido.
- **2**: 100% invertido.

Diccionario de Variables del Entorno

Para comprender la lógica interna del simulador (*environment.py*), se definieron las siguientes variables clave:

- *initial_balance*: Capital inicial asignado al agente ($10,000.00).
- *balance*: Cantidad de efectivo libre (fiat) disponible en la cuenta en un momento dado.
- *shares_held*: Cantidad de acciones del activo (Tesla) que el agente posee actualmente.
- *net_worth*: Patrimonio total del agente en dólares. Se calcula sumando el *balance* en efectivo más el valor actual de las acciones *(shares_held * current_price*).
- *current_position*: Nivel de exposición actual en el mercado, representado como proporción (0.0, 0.5 o 1.0).
- *transaction_fee*: Costo porcentual cobrado por cada operación de compra o venta (0.001 o 0.1%).
- *trade_value*: Diferencia en dólares entre la inversión objetivo (*target_investment*) y la inversión actual (*current_investment*). Un valor positivo indica que se debe comprar; un valor negativo indica que se debe vender.

Ejemplo de Operación Matemática del Agente

Supongamos que estamos en un paso temporal y el agente decide invertir. ¿Cómo calcula el entorno la operación real aplicando las variables anteriores?

Estado inicial del paso:

- Capital (*balance*): $10,000.00
- Acciones (*shares_held*): 0
- Precio de la acción *(current_price)*: $200.00
- Acción elegida por la red neuronal: *1* (que se mapea a *target_position = 0.5*, es decir, 50% del portafolio).

Cálculos del Motor (Step):

1. **Cálculo de Patrimonio (Net Worth):** *net_worth = 10,000 + (0 * 200) = $10,000.*
2. **Inversión Objetivo:** *target_investment = 10,000 * 0.5 = $5,000.*
3. **Inversión Actual:** *current_investment = 0 * 200 = $0.*
4. **Valor a Operar** *(trade_value): 5,000 - 0 = $5,000* (Como es mayor a 0, implica una Compra).
5. **Costo de Transacción (0.1%):** *cost = 5,000 * (1 + 0.001) = $5,005.*
6. **Actualización de Variables:**
    - Nuevo *balance: 10,000 - 5,005 = $4,995.*
    - Nuevas *shares_held: 5,000 / 200 = 25 acciones.*

- Nueva *current_position: 0.5*.

*Nota: Si el agente desea vender (trade_value < 0), la fórmula resta la comisión de las ganancias: proceeds = abs(trade_value) * (1 - transaction_fee).*

2. Fricción de Mercado y Recompensa

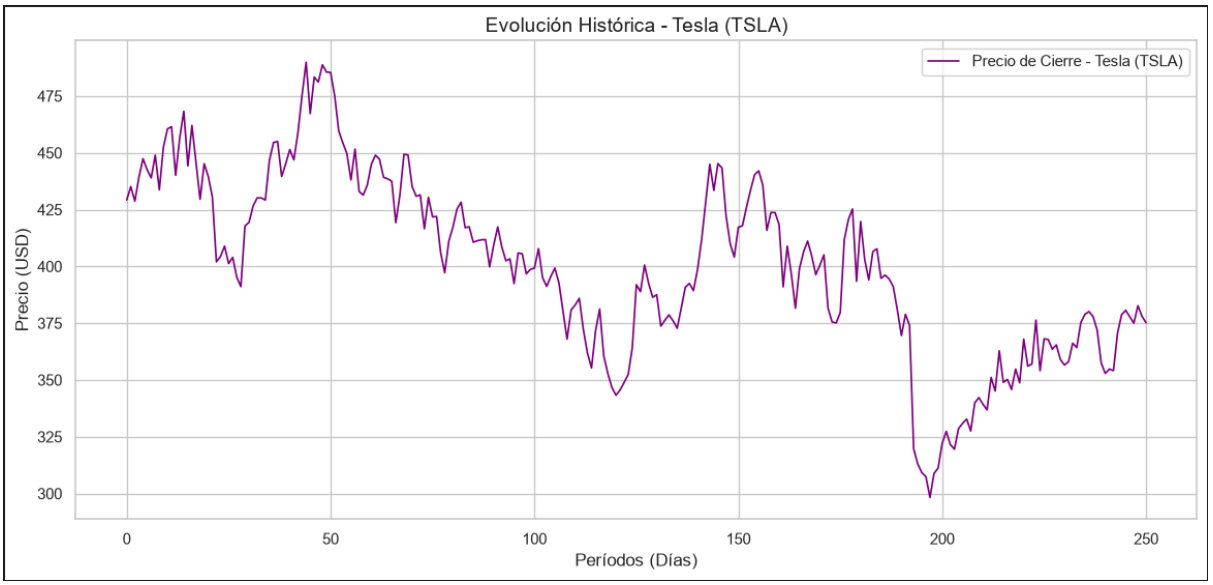
- **Comisiones:** Se implementó una fricción realista del 0.1% por transacción. Esto evita que el agente haga "overtrading" (operar compulsivamente sin sentido).
- **Recompensa (Reward):** Se definió como el cambio porcentual diario del patrimonio neto: *(net_worth - previous_net_worth) / previous_net_worth.*
    - *Reto superado:* Inicialmente, al enfrentar comisiones, el agente adoptó una política excesivamente conservadora (0% invertido siempre) para evitar pérdidas, obteniendo 0.00% de ganancia. Al normalizar correctamente los estados y refinar la recompensa, logró conectar el movimiento del mercado con su crecimiento patrimonial.

3. Entrenamiento del Agente con DQN

Para el entrenamiento del agente se tomó como fuente de datos la base de datos de acciones de Tesla en la pagina de Yahoo Finace:
https://finance.yahoo.com/quote/TSLA/history/

	date	close	high	low	open	volume
0	2025-10-14	429.239990	434.200012	417.859985	426.790009	72669400
1	2025-10-15	435.149994	440.510010	426.329987	434.899994	71558200
2	2025-10-16	428.750000	439.350006	421.309998	434.730011	77189900
3	2025-10-17	439.309998	441.459991	423.600006	425.500000	89331600
4	2025-10-20	447.429993	449.799988	440.609985	443.869995	63719000

```
<class 'pandas.DataFrame'>
RangeIndex: 251 entries, 0 to 250
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   date    251 non-null    str
 1   close   251 non-null    float64
 2   high    251 non-null    float64
 3   low     251 non-null    float64
 4   open    251 non-null    float64
 5   volume  251 non-null    int64
dtypes: float64(4), int64(1), str(1)
memory usage: 11.9 KB
```



- **El Algoritmo:** Utilizamos **DQN (Deep Q-Network)** impulsado por la librería **Stable-Baselines3**, estructurando el proyecto de manera modular con archivos de configuración en formato YAML y un control estricto de experimentos mediante **MLflow**.
- **Optimización (Optuna):** Experimentamos buscando los mejores hiperparámetros *(learning_rate y batch_size)* mediante búsquedas automatizadas con Optuna. Aunque un entrenamiento corto prioriza configuraciones agresivas, se vio la importancia de alinear los pasos de entrenamiento (*timesteps*) a largo plazo (200,000 pasos) para lograr modelos estables.

Para el entrenamiento inicial se tomaron los siguientes hyperparameters:

tesla_dqn.yaml

```yaml
experiment: tesla_dqn_trading
env: Tesla-Daily
algo: dqn
timesteps: 200_000
eval_episodes: 10
hyperparams:
  learning_rate: 0.0001
  buffer_size: 50_000
  batch_size: 32
  learning_starts: 1000
  gamma: 0.99
tune:
  trials: 10
  timesteps: 50000
  search_space:
    learning_rate: {type: float, low: 1.0e-5, high: 1.0e-3, log: true}
    batch_size: {type: categorical, choices: [32, 64, 128]}
```

De acuerdo a lo investigado estos parámetros iniciales tiene la siguiente connotación:

learning_rate: 1e-4: Es una tasa de aprendizaje conservadora y estándar para redes neuronales profundas en RL. Si fuera muy alta (ej. 1e-2), el agente cambiaría sus decisiones de forma tan drástica que desestabilizaría el aprendizaje. Si fuera muy baja, tardaría una eternidad en aprender algo útil.

buffer_size: 50_000: Define cuántas experiencias pasadas recientes recuerda el agente en su memoria. Es un tamaño ideal para almacenar suficientes datos históricos de precios sin saturar la memoria RAM

batch_size: 32: Es el tamaño del grupo de filas que la red neuronal procesa en cada paso de entrenamiento. El número 32 equilibra perfectamente la estabilidad del gradiente y la velocidad de cómputo.

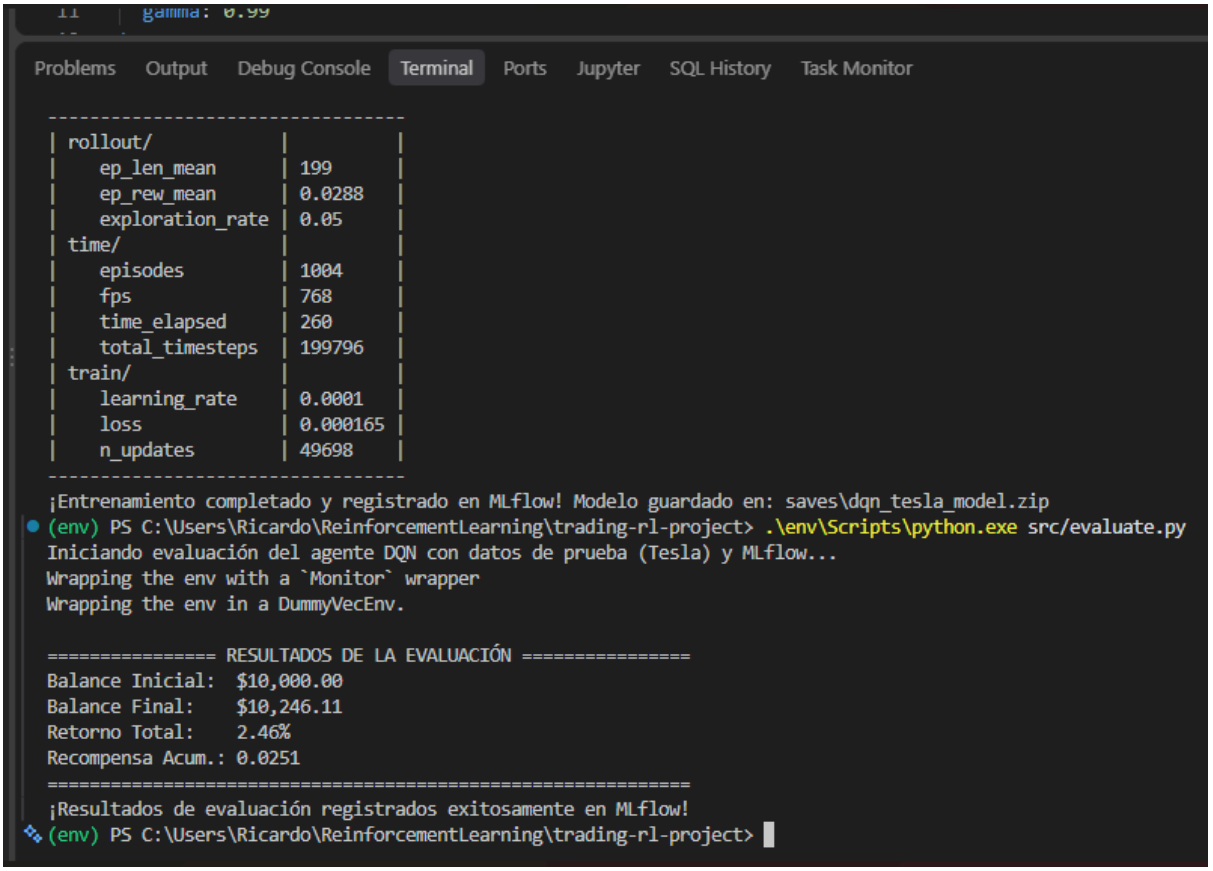
gamma: 0.99: Es el factor de descuento. Un valor cercano a 1 (como 0.99) le indica al agente que debe valorar las recompensas a mediano y largo plazo, lo cual es fundamental en el trading para no obsesionarse solo con una ganancia de un día, sino buscar una estrategia sostenible.

Y se tomaron como parámetros de inicio del entorno de entrenamiento los siguientes:

```
 # 4. Instanciar entorno

       env = TradingEnv(df=df_train, initial_balance=10000.0,
transaction_fee=0.001)
```

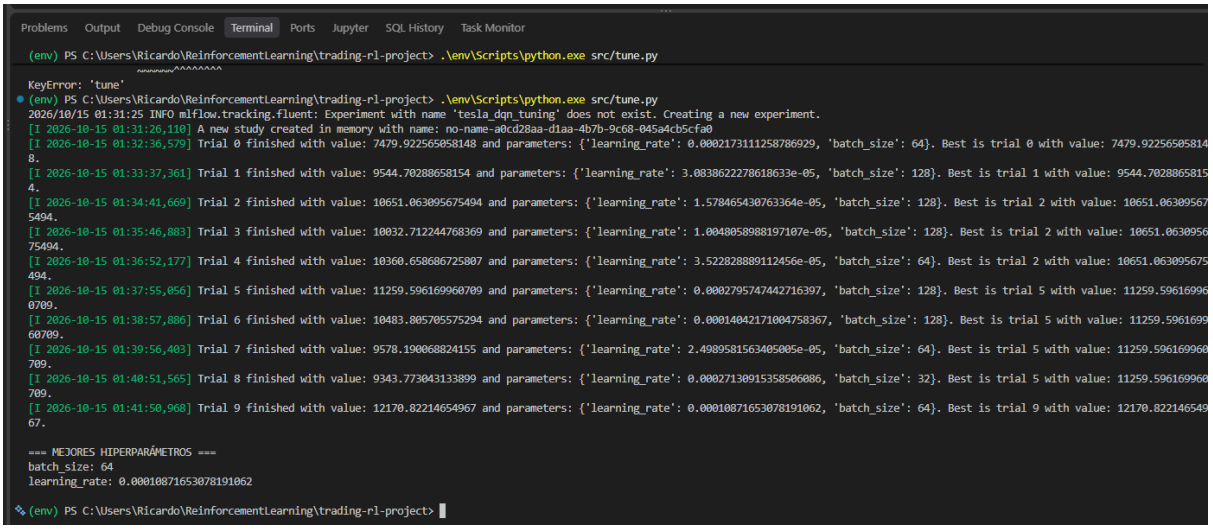
Los resultados obtenidos en primera instancia de acuerdo a la evaluación del modelo con los datos de prueba , fueron los siguientes:



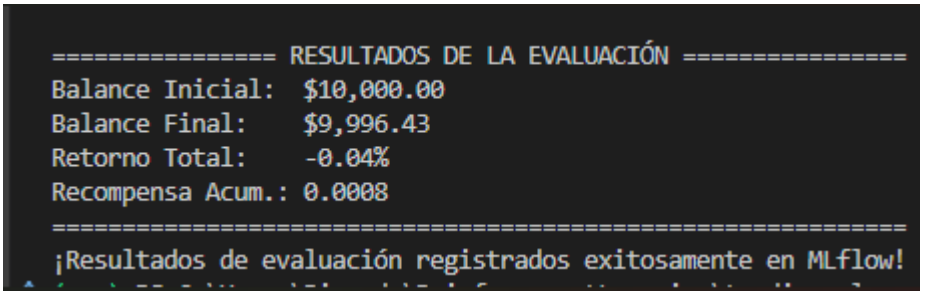
4. Optimización de Hiperparámetros con Optuna

Para extraer el máximo rendimiento del algoritmo DQN impulsado por Stable-Baselines3, implementamos un esquema avanzado de optimización de hiperparámetros automatizado utilizando **Optuna** (*tune.py*).

- **Espacio de Búsqueda:** Evaluamos dinámicamente el *learning_rate* (en escala logarítmica) y el *batch_size* (como variable categórica).
- **Función Objetivo:** Maximizar el balance final (*final_balance*) del portafolio de pruebas.
- **Tracking con MLflow:** Cada "trial" o experimento de Optuna se registró como una ejecución anidada en MLflow (*mlflow.start_run(nested=True)*), permitiendo capturar de forma ordenada las combinaciones probadas y sus respectivas métricas de éxito (balance final).
- **Hallazgo clave:** Si bien las configuraciones agresivas aceleraban el entrenamiento a corto plazo, descubrimos que para lograr estabilidad frente al ruido financiero, fue vital entrenar el modelo durante horizontes prolongados (200,000 timesteps).



Se realizó de nuevo un entrenamiento con estos hiperparametros sugeridos y se obtuvo el siguiente resultado:



Con esta sugerencia se tuvo un retorno inferior de **-0.04%** y un balance de **$9,996.43**, es decir se tuvo pérdida.

De acuerdo a lo investigado se sugiere que pueden existir dos razones posibles para esto:

1. **Sobreajuste (Overfitting) al conjunto de evaluación de entrenamiento:** Optuna califica a los modelos durante sus intentos (*trials*) basándose en la recompensa media que obtienen *durante su propio entrenamiento*. El modelo de la imagen encontró una ruta para verse excelente durante esos 50,000 pasos de sintonización, pero no aprendió patrones reales (generalización). Cuando se evaluó en el conjunto de prueba (*test_tesla.csv*), esos parámetros fracasaron estrepitosamente porque memorizaron el entrenamiento en lugar de aprender a operar.
2. **Tiempo de entrenamiento desigual:** Optuna eligió ese ***learning_rate*** y ***batch_size*** porque fueron los que dieron mejor resultado en una carrera corta (50,000 pasos). Como indica la documentación del repositorio, los "intentos cortos favorecen hiperparámetros que aprenden rápido, los cuales no siempre son los que terminan siendo mejores". Al obligar a esa misma configuración a correr durante 200,000 pasos en el entrenamiento final, el modelo probablemente se "quemó" o sus pesos oscilan de manera perjudicial.

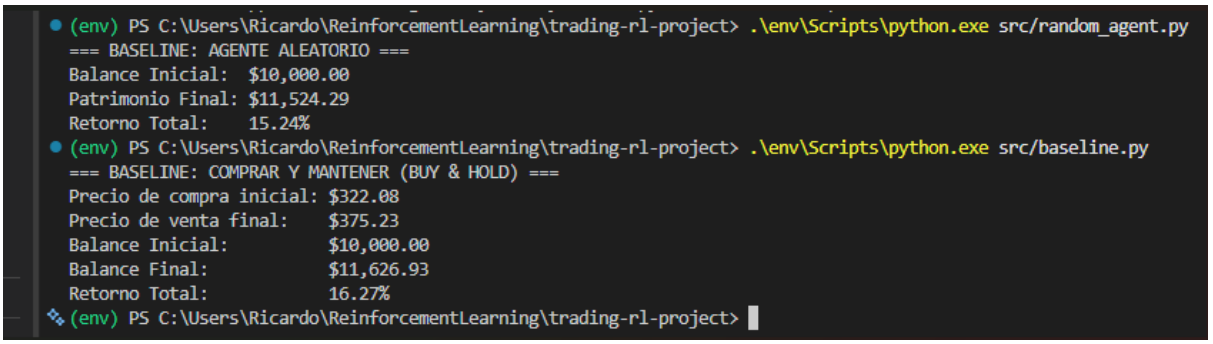
De acuerdo a esto se optó por dejar los hiperparametros originales.

5. Evaluación y Resultados (Datos Hold-out)

El modelo fue puesto a prueba en un conjunto de datos del 20% no visto previamente en el entrenamiento, enfrentándolo a dos estrategias de referencia (Baselines).

Estrategia	Retorno Total	Comportamiento
Buy & Hold	+16.27%	Compra el 100% el primer día y retiene hasta el final.
Agente Aleatorio	+15.24%	Compra, vende o mantiene al azar. Positivo por el sesgo alcista del mercado.
DQN Agent	+2.46%	Dinámico y autónomo (Rinde ~4.5% si se eliminaran los costos de comisión).

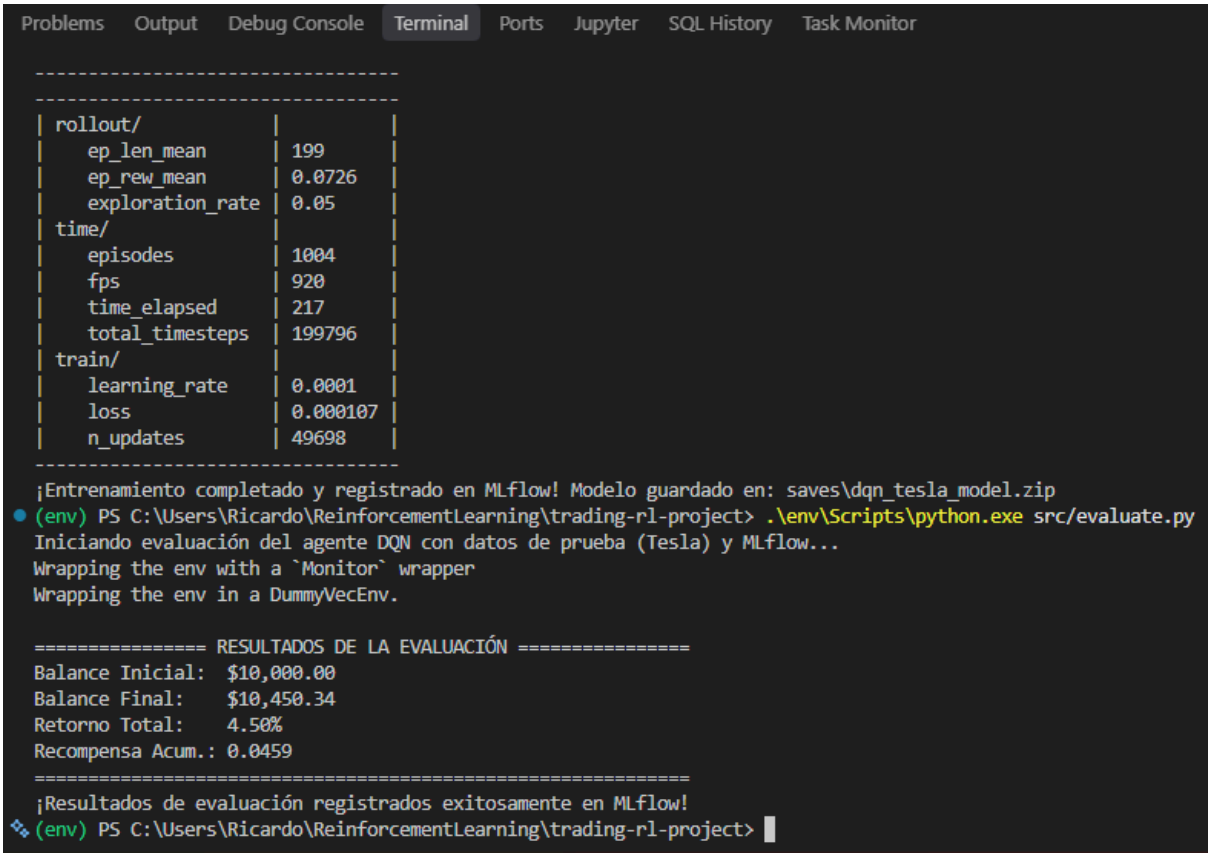
Evaluamos el modelo final en el conjunto de prueba de Tesla y lo contrastamos frente a dos estrategias de referencia (*baselines*):



1. **Comprar y Mantener (*Buy & Hold*):** Comprar todo el primer día y no hacer nada hasta el final. En este periodo de prueba alcista, dio un retorno natural de **16.27%**.
2. **Agente Aleatorio:** Tomar decisiones al azar en cada paso. Obtuvo un **15.24%** debido a la fuerte tendencia alcista general del mercado en esas fechas.
3. **Nuestro Agente DQN:** Tras el entrenamiento, el agente logró romper el estancamiento y generar un rendimiento positivo sólido de **2.46%** (con comisión del

0.001%) y hasta un **4.50%** en pruebas de sensibilidad sin comisiones (0%), demostrando que su política es rentable y autónoma.

En este último punto, se realizó una prueba adicional dejando el costo de comisión en 0 , es decir no tener costos de operación y el agente dio un mejor retorno , esto da a entender que el agente se vuelve más conservador al tener que descontar por las acciones un costo de operación que disminuye el retorno.



6. Discusión y Reflexión del Proceso (Lecciones Aprendidas)

- **¿Por qué el Buy & Hold tuvo mejor retorno al DQN en este experimento?**

  En primera instancia, la estrategia "Comprar y Mantener" parece superior, pero posiblemente no es así. En un mercado altamente alcista como el evaluado, comprar y no hacer nada es óptimo matemáticamente. Además, el *Buy & Hold* sólo paga comisión 2 veces (entrada y salida), mientras que el Agente DQN acumula cobros constantemente por sus gestiones diarias. Esto obligó a la IA a volverse deliberadamente conservadora para no tener demasiada pérdida y por ende se comportó más conservador..

- **El valor de la aversión al riesgo:** Nuestro agente DQN no busca ganar ciegamente, sino aprender a gestionar el capital con prudencia.
- **La importancia de la depuración visual y MLflow:** Tener un registro de métricas y poder visualizar las curvas de aprendizaje ayuda enormemente a determinar cómo se puede ir mejorando el modelo o con que hiperparametros se comportó mejor.
- **El Valor de MLOps (MLflow + Optuna):** El rastreo de experimentos creo que es necesario y no se debe tomar como algo opcional. Trabajar "a ciegas" en RL hace que sea muy complicado si al aplicar mejoras en los hiperparametros estamos mejorando o retrocediendo.